

Chapter 11

程式邏輯入門 - JavaScript 基礎

Horazon

互動媒體設計

為什麼要學 JavaScript?

HTML 是骨架，CSS 是皮膚，JavaScript (JS) 是靈魂。

- 沒有 JS 的網頁：像是一張海報，只能看，不能動。
- 有 JS 的網頁：像是一個應用程式，可以跟你互動。
 - 點按鈕跳出視窗。
 - 檢查表單有沒有填錯。
 - 從伺服器抓取資料 (例如天氣、股價)。
 - 製作網頁遊戲。

JavaScript 是目前世界上最熱門的程式語言，沒有之一。

Hello World

我們怎麼開始寫 JS ？

1. 在 HTML 裡面，用 `<script>` 標籤包起來。
2. 通常放在 `<body>` 的**最下面** (確保 HTML 載入完了才執行 JS)。

```
<body>
  <h1>我的網頁</h1>

  <script>
    alert("Hello World!"); // 跳出警告視窗
    console.log("你好，控制台！"); // 在 Console 印出訊息
  </script>
</body>
```

Console 在哪？

按 **F12** 開發者工具 -> 切換到 **Console** 分頁。

變數 - 記憶體的房子

變數就是用來存資料的房子。

宣告方式 (ES6 標準)：

1. `let`：一般的變數，數值**可以**改變。

- `let score = 100;`

- `score = 90;` (OK!)

2. `const`：常數，數值**不能**改變。

- `const pi = 3.14;`

- `pi = 3.14159;` (Error! 會報錯)

`var` 是舊時代的寫法，盡量少用 (因為它的作用域很混亂)。

資料型別

JS 裡的資料分幾種：

1. Number (數字)：10, 3.14, -5。
2. String (字串)：用引號包起來的文字。
 - "Hello", '你好', `模板字串`。
3. Boolean (布林)：只有兩種，true (真) 或 false (假)。
4. Array (陣列)：一排盒子，存多個資料。
 - `let fruits = ["Apple", "Banana", "Orange"];`
5. Object (物件)：有多個屬性的複雜東西。
 - `let student = { name: "John", age: 18 };`

運算子

- 算術運算：`+`，`-`，`*`，`/`，`%` (取餘數)。
 - 注意：字串相加是「串接」。
 - `"10" + 20` 變成 `"1020"` (不是 30 喔！)。
- 比較運算：
 - `>` (大於)，`<` (小於)，`>=` (大於等於)。
 - `===` (嚴格等於)：數值和型別都要一樣。
 - `!==` (不等於)。
 - 盡量不要用 `==` (寬鬆等於)，它會亂轉型。
- 邏輯運算：
 - `&&` (AND, 且)：兩邊都要對，才是對。
 - `||` (OR, 或)：只要有一邊對，就是對。
 - `!` (NOT, 非)：把對變錯，錯變對。

條件判斷 (If...Else)

讓程式有「思考」的能力。

```
let score = 85;

if (score >= 60) {
  console.log("及格!");
} else {
  console.log("被當了 QQ");
}
```

還可以有多個條件：

```
if (score >= 90) { ... }
else if (score >= 80) { ... }
else { ... }
```

迴圈

讓程式幫你做重複的事。

For 迴圈 (固定次數)

```
// 印出 0 到 4
for (let i = 0; i < 5; i++) {
  console.log("第 " + i + " 次執行");
}
```

陣列迴圈 (遍歷資料)

```
let fruits = ["Apple", "Banana", "Orange"];

fruits.forEach(function(item) {
  console.log("我喜歡吃 " + item);
});
```


函式 - 打包程式碼

把一堆程式碼包成一個指令，隨時可以呼叫。

定義函式：

```
function sayHello(name) {  
    console.log("你好，" + name + "！");  
}
```

呼叫函式：

```
sayHello("小明"); // 印出：你好，小明！  
sayHello("阿華"); // 印出：你好，阿華！
```

參數：括號裡的 `name`，是我們傳進去的資料。

實作練習：BMI 計算機 (邏輯版)

請在 Console 裡寫一個計算 BMI 的程式：

1. 定義變數 `height` (身高，公尺) 和 `weight` (體重，公斤)。
2. 計算 $BMI = \text{體重} / (\text{身高} * \text{身高})$ 。
3. 使用 `if...else` 判斷：
 - $BMI < 18.5$: "過輕"
 - $18.5 \leq BMI < 24$: "正常"
 - $BMI \geq 24$: "過重"
4. 用 `console.log` 印出結果。

```
let h = 1.75;  
let w = 70;  
// ... (請自己寫寫看)
```

補充：陣列的常用操作

陣列 (Array) 在 JS 裡面非常常用，我們常常需要新增或刪除裡面的資料。

- `length`：取得陣列的長度（有幾個資料）。
- `push()`：把資料塞進陣列的**最後面**。
- `pop()`：把陣列**最後面**的資料拿出來。

```
let fruits = ["Apple", "Banana"];  
console.log(fruits.length); // 印出 2
```

```
fruits.push("Orange"); // 變成 ["Apple", "Banana", "Orange"]  
let last = fruits.pop(); // last 會變成 "Orange"，陣列變回原本的樣子
```

補充：寫程式的好習慣

寫程式不是只要「能動」就好，還要讓自己和別人「看得懂」。

1. 加上註解：

- 單行註解：`// 這是一行註解`
- 多行註解：`/* 這是多行註解 */`

2. 變數命名規則 (小駝峰式 Camel Case)：

- 第一個單字小寫，後面的單字第一個字母大寫。
- 例如：`myFirstName`，`totalScore`，`btnSubmit`。

3. 適當的縮排：讓程式碼有階層感，不會像是一團亂碼。

補充：字串的常用操作

字串不只能用來顯示文字，還有很多內建的工具可以使用。

- `length`：取得字串的長度。
- `toUpperCase()`：把所有英文字母變成大寫。
- `replace()`：替換字串裡面的文字。

```
let text = "Hello JavaScript!";  
console.log(text.length); // 印出 17  
  
console.log(text.toUpperCase()); // "HELLO JAVASCRIPT!"  
console.log(text.replace("Hello", "Hi")); // "Hi JavaScript!"
```

補充：物件 (Object) 的基本操作

當我們想要把多個相關的資料綁在一起時，就會使用「物件」。
像是描述一個學生的資料：

```
let student = {  
  name: "小明",  
  age: 18,  
  isMale: true  
};  
  
// 取得物件裡面的資料：使用「點 (.)」  
console.log(student.name); // 印出 "小明"  
  
// 修改物件裡面的資料  
student.age = 19;  
console.log(student.age); // 印出 19
```

補充：Math 數學函式

JS 內建了一個 `Math` 工具箱，可以幫我們做各種數學計算。

- `Math.round()`：四捨五入。
- `Math.ceil()`：無條件進位。
- `Math.floor()`：無條件捨去。
- `Math.random()`：產生一個 0 到 1 之間的隨機小數 (不包含 1)。

```
console.log(Math.round(3.6)); // 印出 4
console.log(Math.floor(3.9)); // 印出 3

// 產生 0 ~ 9 的隨機整數
let randomNum = Math.floor(Math.random() * 10);
```

總結：JavaScript 核心觀念

恭喜你完成了 JavaScript 的基礎！

這幾天我們學到了：

1. **變數與資料型別**：儲存各種資料 (數字、字串、布林)。
2. **運算子與條件判斷**：讓程式學會算數與「做決定 (`if...else`)」。
3. **迴圈**：讓程式不知疲倦地重複做事 (`for`)。
4. **函式與陣列/物件**：把程式碼與資料整理得井然有序。

多寫、多錯、多查資料，這就是成為工程師的必經之路！